

Assignment 3: RAG for Science Question Answering

Student ID: 314832008 **Source Code:** <https://github.com/tingul4/RNN-course>

1. Overview

In this assignment, I built a full Retrieval-Augmented Generation (RAG) pipeline to answer multiple-choice science questions from the Kaggle LLM Science Exam dataset. The pipeline covers three stages: (1) data chunking and indexing, (2) two-stage retrieval with vector search and cross-encoder re-ranking, and (3) answer generation using a local LLM.

Experimental Setup

The knowledge base was constructed directly from the provided `train.csv` dataset, which was downloaded from the official [Kaggle LLM Science Exam competition](#) and contains 200 science-related multiple-choice questions. Each question has five answer choices (A–E). To facilitate effective retrieval, I transformed the dataset by treating each `(question_prompt + one answer option)` pair as a separate, individual document. This transformation creates a corpus of **1,000 documents** covering diverse STEM topics. By querying the system with the pure question prompt, we can objectively measure the retrieval quality (Recall@3) by checking whether the document containing the correct answer option is successfully retrieved.

Models used:

- Embedding: `BAAI/bge-m3` (CUDA)
 - Re-ranker: `cross-encoder/ms-marco-MiniLM-L-6-v2` (CUDA)
 - Generator: `llama3:8b` (via Ollama)
-

2. Part 1 — The Indexing Pipeline

2.1 Chunking Strategies

I implemented two distinct chunking strategies:

- **Method A (Fixed-Size Chunking):** Used naive fixed-size chunking via `RecursiveCharacterTextSplitter`. I used `chunk_size=500` with a standard `chunk_overlap=50`. This establishes a baseline for retrieving raw data block by block based on character count.
 - **Method B (Semantic Chunking):** Utilized LangChain's `SemanticChunker` with the `BAAI/bge-m3` embedding model. Instead of splitting by character count, this method dynamically splits text based on semantic similarity between sentences, aiming to keep related concepts together and break at topic shifts.
-

3. Part 2 — Retrieval & Re-ranking

3.1 Setup

- **Stage 1 (Dense Retrieval):** Vector similarity search with `k=20` candidates.
- **Stage 2 (Re-ranking):** Cross-encoder scores each (query, candidate) pair; top-3 are kept as the final context.
- **Baseline:** Vector-only retrieval returning top-3 directly (no cross-encoder).
- **Metric:** Recall@3 — whether the correct answer option appears in the top-3 results.

Experiments were run on the first 50 training questions.

3.2 Hit Rate Results

Configuration	Index A (Fixed, size=500)	Index B (Semantic Chunker)
Vector Search Only	0.60	0.58
Vector Search + Rerank	0.70	0.62
Improvement	+0.10 (+10%)	+0.04 (+4%)

(Note: Unlike long-form documents, the Kaggle dataset consists of short, self-contained Q/A pairs. The `SemanticChunker` (Index B) dynamically splits text based on sentence-level semantic shifts, which occasionally over-fragments a single long answer option and breaks its context. On the other hand, the Fixed chunking strategy (Index A) safely wraps the short Q/A pairs within its character limit, preserving their integrity. Consequently, naive fixed chunking yielded slightly better retrieval performance for this specific dataset.)

4. Part 3 — Generation with Local LLM

4.1 Setup

- **LLM Integration:** Connected the end-to-end pipeline to a local LLM API server running `llama3:8b` via Ollama. The top-3 context chunks generated from the Stage 2 cross-encoder were concatenated and fed into the prompt.
- **System Prompt Design:** A strict system prompt was utilized to actively mitigate hallucinations. The LLM was instructed: *"You are a helpful science assistant. Answer the question based ONLY on the context provided below. If the answer is not in the context, say 'I don't know'."*

4.2 Accuracy Results

- **Evaluation Dataset:** Ran the full RAG pipeline on an evaluation set holding 50 generic science questions from the main Kaggle dataset.
- **Accuracy Score:** The pipeline successfully deduced the correct options around ~70% of the time on average (`Accuracy = 0.70`). This firmly matches the optimal Recall@3 (0.70) from Stage 2 retrieval, demonstrating that the LLM accurately synthesizes context to pick choices when the correct information is present in the re-ranked chunks.

5. Discussion

5.1 Chunk Size Analysis

I evaluated the impact of different chunk sizes by comparing `chunk_size=500` and `chunk_size=1000` using the Fixed chunking strategy (with Stage 1 Vector Search only) on the query: *"What generates energy in the cell?"*

- **Chunk Size = 500 (Score ~0.85):** The system retrieved chunks related to "power density in energy systems" and "renewable energy." It matched the keyword "energy" but completely missed the biological context ("in the cell"). Furthermore, all top three retrieved chunks were identical. This indicates that a smaller chunk size over-fragments the text, likely severing the question prompt from its corresponding answer choices, resulting in redundant and context-poor semantics.
- **Chunk Size = 1000 (Score ~0.92-0.93):** The system accurately retrieved relevant documents discussing "active and passive transport in cells" and "energy input from the cell." The larger chunk size successfully encapsulated the entire question prompt alongside its options within the same chunk. This preserved the crucial semantic relationship between "energy" and "cells," yielding highly relevant hits.

Conclusion: A `chunk_size` of 1000 is significantly more effective for this dataset. Smaller chunks (e.g., 500 characters) suffer from context loss and fragmentation, causing the embedding model to focus on isolated keywords (like "energy" in physics) rather than the complete conceptual meaning.

5.2 Re-ranking Impact

The re-ranking stage plays a critical role in overcoming the limitations of standard vector search, especially for complex scientific queries where exact conceptual relationships matter more than keyword overlap. Here are two examples demonstrating this:

Example 1 — ID#3 (What is the significance of regularization in terms of renormalization problems in physics?, correct = C)

- **Stage 1 (Vector Search):** Initially, the dense vector search prioritized documents with broad keyword overlaps. It retrieved texts discussing "regularization" in machine learning contexts (like L1/L2 regularization) or generic physics problems at the top, pushing the correct physics-specific document down in the top-20.
- **Stage 2 (Re-ranking):** The cross-encoder processes the query and document together to evaluate deep semantic relationships. It correctly identified the specific theoretical physics context (the intersection of "regularization" and "renormalization") and successfully promoted the correct document (Option C) straight to the top.

Example 2 — ID#11 (What is the relationship between the Wigner function and the density matrix operator?, correct = A)

- **Stage 1 (Vector Search):** The vector search retrieved several irrelevant documents that mentioned "Wigner function" or "density matrix" in isolation, leaving the correct answer out of the top results. It struggled to evaluate the nuanced relationship being asked for.
- **Stage 2 (Re-ranking):** The cross-encoder effectively interpreted the relational aspect of the query ("What is the relationship between..."). It scanned the initial top-20 candidates and bumped the correct document (Option A) to the #1 position because it explicitly explained the mathematical mapping between the two concepts rather than just randomly hitting the keywords.

5.3 Latency

- **Vector Search:** ~0.02 seconds (Note: The first query incurs a cold-start overhead of ~0.45 seconds due to PyTorch/ONNX model and index initialization, but subsequent queries consistently drop to ~20 milliseconds).
- **Re-ranking (Cross-Encoder):** ~0.015 to 0.08 seconds. Although cross-encoders are generally computationally expensive, the latency remains very low because it only processes a small subset of documents (the top-20 candidates) retrieved from Stage 1.
- **LLM Generation:** ~0.65 seconds. The auto-regressive text generation by the local LLM is the most time-consuming stage of the pipeline, as it reads the context and predicts the response token by token.
- **Is the re-ranking worth the extra time cost?** Yes, absolutely. The re-ranking stage only adds a minimal latency overhead (around 15-80 milliseconds) to the overall pipeline. Compared to the LLM text generation stage, which takes nearly an order of magnitude longer (~0.65+ seconds), this cost is negligible. In exchange for this marginal delay, the cross-encoder significantly improves the precision of the context fed into the LLM. This high-quality context strongly enhances the final answer accuracy and effectively reduces hallucinations, making the trade-off highly worthwhile.